

Создание списков

Для того, чтобы создать список состоящий из 1010 нулей мы можем использовать следующий код:

```
zeros = []  
  
for i in range(10):  
    zeros.append(0)
```

В Python, однако есть более простой и компактный способ для создания такого списка. Мы можем использовать оператор умножения списка на число:

```
zeros = [0] * 10
```

Для создания списков, заполненных по более сложным правилам нам приходится явно использовать цикл `for`.

Например, для создания списка целых чисел от 00 до 99, мы вынуждены писать такой код:

```
numbers = []  
  
for i in range(10):  
    numbers.append(i)
```

Такой код хоть и не является сложным, однако достаточно громоздок.

Списочные выражения

В Python есть механизм для создания списков из неповторяющихся элементов. Такой механизм называется — **списочное выражение** (list comprehension).

Предыдущий код можно записать следующим образом:

```
numbers = [i for i in range(10)]
```

Общий вид списочного выражения следующий:

```
[выражение for переменная in последовательность]
```

где `переменная` — имя некоторой переменной, `последовательность` — последовательность значений, которые она принимает (список, строка или объект, полученный при помощи функции `range`), `выражение` — некоторое выражение, как правило, зависящее от использованной в списочном выражении переменной, которым будут заполнены элементы списка.

Примеры использования списочных выражений

1. Создать список, заполненный 10 нулями можно и при помощи списочного выражения:

```
zeros = [0 for i in range(10)]
```

2. Создать список, заполненный квадратами целых чисел от 0 до 9 можно так:

```
squares = [i ** 2 for i in range(10)]
```

3. Создать список, заполненный кубами целых чисел от 10 до 20 можно так:

```
cubes = [i ** 3 for i in range(10, 21)]
```

4. Создать список, заполненный символами строки:

```
chars = [c for c in 'abcdefg']
```

```
print(chars)
```

Считывание входных данных

При решении многих задач из предыдущих уроков мы считывали начальные данные (строки, числа) и заполняли ими список. С помощью списочных выражений процесс заполнения списка можно заметно сократить.

Например, если сначала вводится число `n` – количество строк, а затем сами строки, то создать список можно так:

```
n = int(input())  
lines = [input() for _ in range(n)]
```

Можно опустить описание переменной `n`:

```
lines = [input() for _ in range(int(input()))]
```

Если требуется считать список чисел, то необходимо добавить преобразование типов:

```
numbers = [int(input()) for _ in range(int(input()))]
```

Обратите внимание, мы используем символ `_` в качестве имени переменной цикла, поскольку она не используется.



Списочные выражения часто используются для инициализации списков. В Python не принято создавать пустые списки, а затем заполнять их значениями, если можно этого избежать.

Условия в списочном выражении

В списочных выражениях можно использовать условный оператор. Например, если требуется создать список четных чисел от 0 до 20, то мы можем написать такой код:

```
evens = [i for i in range(21) if i % 2 == 0]
```



Важно: для того, чтобы получить список, состоящий из четных чисел, лучше использовать функцию `range(0, 21, 2)`. Предыдущий пример приведен для демонстрации возможности использования условий в списочных выражениях.

Вложенные циклы

В списочном выражении можно использовать вложенные циклы.

Следующий программный код:

```
numbers = [i * j for i in range(1, 5) for j in range(2)]  
print(numbers)
```

выведет список:

```
[0, 1, 0, 2, 0, 3, 0, 4]
```

Такой код равнозначен следующему:

```
numbers = []  
  
for i in range(1, 5):  
    for j in range(2):  
        numbers.append(i * j)  
  
print(numbers)
```

Подводя итог

Пусть `word = 'Hello'`, `numbers = [1, 14, 5, 9, 12]`, `words = ['one', 'two', 'three', 'four', 'five', 'six']`.

Списочное выражение	Результирующий список
<code>[0 for i in range(10)]</code>	<code>[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]</code>

Списочное выражение	Результирующий список
<code>[i ** 2 for i in range(1, 8)]</code>	<code>[1, 4, 9, 16, 25, 36, 49]</code>
<code>[i * 10 for i in numbers]</code>	<code>[10, 140, 50, 90, 120]</code>
<code>[c * 2 for c in word]</code>	<code>['HH', 'ee', 'll', 'll', 'oo']</code>
<code>[m[0] for m in words]</code>	<code>['o', 't', 't', 'f', 'f', 's']</code>
<code>[i for i in numbers if i < 10]</code>	<code>[1, 5, 9]</code>
<code>[m[0] for m in words if len(m) == 3]</code>	<code>['o', 't', 's']</code>